

BRUNEI CYBER SECURITY ASSOCIATION



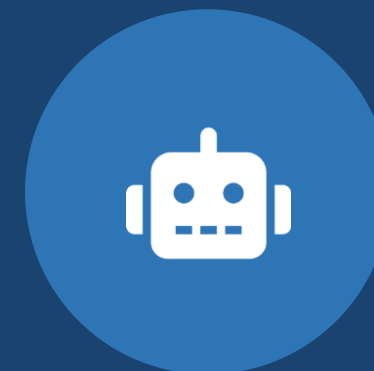
Event wi-fi:
STACK2026 🤖
m@ngk@11-br@d@

Agentic Coding 101

Agents and Skills with Codex

Hands-on lab · **Beginner** · **Runs in Docker**

Build three small apps, and feel the difference agents and skills make.





A bit about me

- Principal Consultant @ Swarmnetics
- Co-founder of cyber673
- Committee Member of BCSA & brunei4AI
- On an ongoing journey of upskilling people through community involvements



**BRUNEI
CYBERSECURITY
ASSOCIATION**

Bringing together professionals, experts, and enthusiasts in cybersecurity to collaborate, share knowledge, and collectively address the evolving challenges of cyber threats.

Our aims

- Promote cybersecurity awareness
- Facilitate skill development
- Encourage collaboration
- Advocate policy & regulation

Become a member: <https://bcsa.bn>



About cyber673



cyber673

A team of cybersecurity enthusiasts, professionals, and educators growing local capability in offensive and defensive security, and the community around it.

What we do

- Monthly meetups on cybersecurity topics
- Hands-on workshops (web, mobile, API security)
- Capture The Flag (CTF) competitions
- Community networking & knowledge sharing

Powered by Swarmnetics, Nitro Security, SPI-BN · Supported by BCSA

More info: <https://cyber673.com>



HOW THE DAY RUNS

Today's Agenda

	Welcome & framing	GenAI and LLM basics, and what agentic coding really is
	The agentic toolkit	Mainstream harnesses: Claude Code, Codex, Gemini CLI, and more
	Secure by default?	Where prompting holds up, and where it quietly breaks
	Agents & Skills 101	What each is, how platforms differ, where files live
	Lab 1 — Pantry	Build it with plain prompting. The baseline
	Break	A short breather
	Lab 2 — Agents & Skills	Rebuild the pantry the agentic way
	Rapid Build round	Reuse the toolkit: Bekantan Run + Makan Mana (Labs 3 & 4)
	Wrap-up	Side by side, the compounding payoff, Q&A

01

SEGMENT

Welcome & Framing



First, the basics: what GenAI and LLMs are. Then what "vibecoding" actually is when you look closely.



START HERE

GenAI & LLMs, the Basics



What's an LLM?

A large language model trained on huge amounts of text. At its core it does one thing: predict the next token. Prediction at scale.



Tokens & context

It reads and writes in tokens (word pieces). The context window is its working memory, and when it fills up it forgets earlier context.



Generative AI

Models that produce new content: text, code, images, audio. LLMs are the text-and-code kind we use here.



Strengths & limits

Great at patterns and drafts. But it can be confidently wrong (hallucinate), and forgets everything between sessions. Non-deterministic.



FROM CHATBOT TO COLLABORATOR

The Agentic Loop

Agentic coding is not one prompt and hope. You give the model tools, context, and standing instructions, then let it work in a loop.



Tools

Files, terminal, tests. The hands it acts with.



Context

Your repo, docs, and conventions. What it knows.



Instructions

Standing rules it follows on every task.



The loop

Plan, act, observe, correct. Repeat until done.

Key shift: the model keeps going on its own until the task is done, not stopping after one reply.

Your job: direct it and review the result. You don't have to write the code yourself.

02

SEGMENT

The Agentic Toolkit



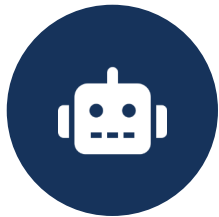
The model is the engine. The harness is the car around it: tools, context, and the loop. Let's meet the mainstream ones.



WHAT RUNS THE LOOP

Mainstream Agentic Harnesses

A "harness" wraps a model with file access, a terminal, and the agentic loop. Same idea, different ecosystems. We use Codex today; the concepts transfer.



Claude Code

Anthropic

CLI-first agent. Strong subagents and Agent Skills.



Codex

OpenAI

CLI, IDE, and cloud. What we build with today.



Gemini CLI

Google

Open-source CLI agent with subagents and skills.

And many more: Cursor, GitHub Copilot, Aider, opencode, Windsurf, Amp, goose, Zed. They differ in interface and defaults, but all share the same loop, and increasingly the same file formats.

03

SEGMENT

Secure by Default, or Just Secure Enough?

Where simple prompting holds up, and where AI-generated code quietly falls short.





A PRAGMATIC LOOK

Holds Up vs. Falls Short



Where it holds up

- ✓ Throwaway scripts and quick prototypes
- ✓ Boilerplate you will read before running
- ✓ Low-risk, well-known patterns
- ✓ When a human reviews every line



Where it falls short

- ✗ Validation and access control
- ✗ Handling secrets and user data
- ✗ Trusting input it should not trust
- ✗ Code shipped without a review

Two AI-specific risks we will hit in the labs



Prompt injection. Text the model reads can carry instructions that hijack it.



Don't trust model output. Validate what it returns before it touches your database.

04

SEGMENT

Agents & Skills 101



What each one is, how they differ, and how the platforms organise their files.



What is an Agent?



A specialised assistant

A focused teammate you brief once and reuse, with its own job, instructions, and tools.

Heads-up: in tool terms these are "subagents" — separate from the one main assistant you chat with.



Scope

One clear job. "Write backend code," not "do everything."



System prompt

Standing instructions: its role, rules, and style.



Tools

What it can touch: files, terminal, tests.



What is a Skill?



A defined task

"Turn a one-liner into a full PRD (a short build spec)," for example.



Step-by-step instructions

The how: the procedure the agent follows.



Invoked on demand

Loaded only when relevant, keeping the agent lean.



A reusable instruction pack

A documented procedure an agent pulls in on demand. Write it once, reuse it everywhere.



WHEN TO REACH FOR WHICH

Agents vs. Skills

	Agent (subagent)	Skill
What it is	A specialised assistant (a who)	A reusable instruction pack (a how)
Analogy	A teammate with a role	A playbook on the shelf
Holds	Scope, system prompt, tools	A procedure for one task
Active when	Invoked for its kind of work	Pulled in only when relevant
You create one to	Own a kind of work end to end	Standardise a recurring task

Rule of thumb: agents own work, skills package know-how. They are strongest together.



When a Prompt Becomes a Skill

Typed roughly the same instructions three times? Stop. Promote it into a skill.



1. One-off prompt

You write a detailed prompt to get a task done once.



2. You repeat it

Same prompt, new project. You are copy-pasting it.



3. Promote to a skill

Save the procedure once, with a clear trigger.



4. Reuse forever

Any agent invokes it on demand. Consistent every time.

Why it matters: skills turn your best prompts into team assets. Quality stops depending on who is typing, and starts depending on the playbook.



DON'T CONFUSE THE TWO

Main File vs. Subagent File

Main instruction file

`CLAUDE.md` · `AGENTS.md` · `GEMINI.md`

Always loaded. Standing context for the ONE main assistant you chat with.
Sets project rules, conventions, and how it should behave.

Think: *the house rules everyone follows.*

Subagent file

`.claude/agents/` · `.codex/agents/` · `.gemini/agents/`

Defines a SEPARATE, scoped assistant: its own system prompt, tools, often its own model, and a fresh context window. Invoked on demand; returns a summary.

Think: *a specialist you call in for a job.*

The trap: people pile everything into the main file. Standing project rules go in the main file; a distinct specialist with its own tools and context goes in a subagent file.



WHERE THEY LIVE, CONCRETELY

A Specialist Agent on Disk

Inside the agents-and-skills workspace: each specialist is one file, each skill is a folder, and the apps build under projects/.

agents-and-skills/

```
|-- AGENTS.md
|-- .codex/agents/
|   |-- coding-agent.toml
|   `-- frontend-design-agent.toml
|-- .agents/skills/
|   |-- prd-expander/
|   |-- ui-mockup/
|   `-- security-review/
`-- projects/
    |-- pantry, game, makan-vote
```



AGENTS.md

The main file: standing house rules for the whole project.



.codex/agents/

Your specialist agents, one TOML file per agent.



.agents/skills/

Your skills, a folder with a SKILL.md for each.

Example layout (Codex). Claude Code uses .claude/agents/ (Markdown); Gemini CLI uses .gemini/agents/.



WHAT'S INSIDE ONE

Inside a Specialist Agent

A subagent file is just an id, when to use it, and its standing instructions. Codex format (TOML) shown.

`coding-agent.toml`

```
name = "coding_agent"
description = "Backend Flask engineer; models, routes, tests."
sandbox_mode = "workspace-write"

developer_instructions = """
You own the backend of small Flask apps.
- Build the model, routes, tests, and Docker.
- Parameterise SQL; validate every input.
- Never hardcode secrets; read keys from env.
"""
```



name

How Codex refers to and spawns this agent.



description

When to use it. Codex matches tasks to this.



developer_instructions

Its system prompt: role, scope, and rules.

model and sandbox_mode are optional.

Example only, fill in your own role and rules. The starter repo generates these files for you.



WHERE THEY LIVE, CONCRETELY

A Skill on Disk

Each skill is a folder: a SKILL.md spec, reference docs the agent pulls in on demand, and an executable helper. A real base to take home.

```
.agents/skills/
```

```
`-- security-review/
    |-- SKILL.md
    |-- reference/
    |   |-- checklist.md
    |   |-- report-format.md
    |-- scripts/
        |-- scan.py
```



SKILL.md

Required. The spec: frontmatter plus the procedure.



reference/

Docs the agent pulls in on demand (checklist, report format).



scripts/

An executable helper (e.g. scan.py, a grep-based smell detector).

Example layout. The same SKILL.md folder works across Claude Code, Codex, and Gemini CLI.



WHAT'S INSIDE ONE

Inside a Skill

A *SKILL.md* is frontmatter (name + description) plus a Markdown body: the steps the agent follows.

security-review/SKILL.md

```
---
name: security-review
description: Review code against a
  checklist before commit.
---

# Security Review

Report each finding as Issue / Risk / Fix:
1. Prompt injection: data, not commands?
2. Validate model output before storing.
3. Server-side input validation.
4. Parameterised SQL; escaped output.
5. Secrets from env, never hardcoded.
```



name

The skill's id.



description

When to use it. The agent matches tasks to this line.



the body

The procedure: the steps the agent follows when it runs.

Progressive disclosure: only name + description load until the skill is actually used.

Example only, fill in your own steps. The starter repo generates these files for you.



THEY COMPOSE

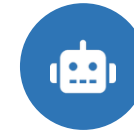
Skills and Agents, Together

Skills and agents are not separate worlds. Each can reach for the other, on its own.



An agent calls a skill

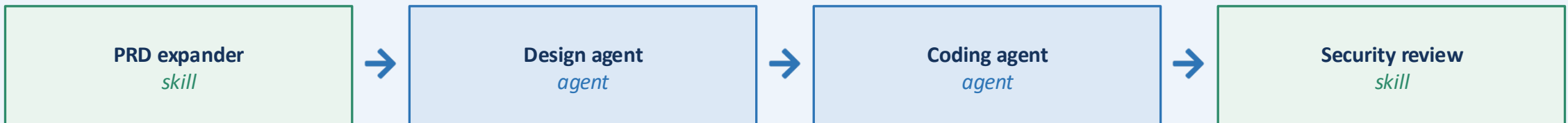
The coding agent reaches for the security-review skill on its own, before it commits. No human prompt needed.



A skill calls an agent

A skill can hand work to a specialist: a "new feature" skill drives the design and coding agents, then runs security-review.

Chain them and you get a pipeline, which is exactly your Lab 2 flow:



05

SEGMENT

Lab 1 — Pantry, with Plain Prompting



Workspace: codex-only/. Build a pantry tracker the simple way, our baseline. You write the Docker too. Note what feels fragile.



BEFORE WE BUILD

Your Workspace

Two workspaces, one per lab style. You talk to Codex in plain English; it builds the app, and each app gets its own Docker.

terminal

```
$ git clone https://github.com/bcsabn/stack-workshop
$ cd agents-and-skills # codex-only for Lab 1
> build a pantry tracker (you, to Codex)
...Codex builds projects/pantry + Docker...

$ cd projects/pantry
$ docker compose up
-> http://localhost:5000
```



docker per app

Build, then `cd projects/<name> && docker compose up`.



Codex

The AI assistant you talk to in plain English.



two workspaces

codex-only (Lab 1) and agents-and-skills (Lab 2+).

Quick vocab

Docker a sealed sandbox: everyone runs the same setup, and the AI can't touch your real machine.

Repo your project folder. **Commit** save a checkpoint of it. **PRD** a short spec of what to build.



The Baseline Build

Your mission

In **codex-only**, build a pantry tracker using nothing but plain prompts. You write the Docker too.

It should let you:

- Add an item with an expiry date
- List what you have
- Flag items expiring soon
- Mark items used or thrown out
- Keep data between restarts

As you build, watch for



Re-explaining yourself

How often you restate context the model forgot.



Quiet shortcuts

Validation or edge cases it silently skipped.



Review burden

How much you must read to trust the output.



Inconsistency

Style and structure drifting as it grows.



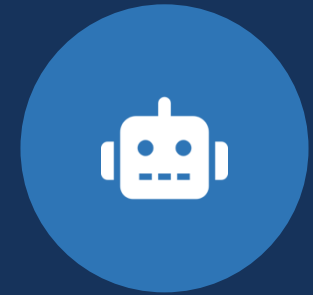
Break

A short breather. Make sure your container is still running before Lab 2.

06

SEGMENT

Lab 2 — Rebuild with Agents & Skills



*Workspace: agents-and-skills/. Same pantry, the agentic way. Two agents, three skills.
Watch the workflow change from typing to directing.*



Two Agents, Three Skills

THE AGENTS (SUBAGENTS)



Coding agent

Backend: data model, logic, tests, AI bits, and the app's own Docker.



Frontend design agent

UI: Jinja2 templates, Tailwind, every state, to an anti-slop bar.

THE SKILLS



PRD expander

Vague request to a one-page PRD before any code.



UI mockup generator

PRD to a static mockup.html for sign-off first.



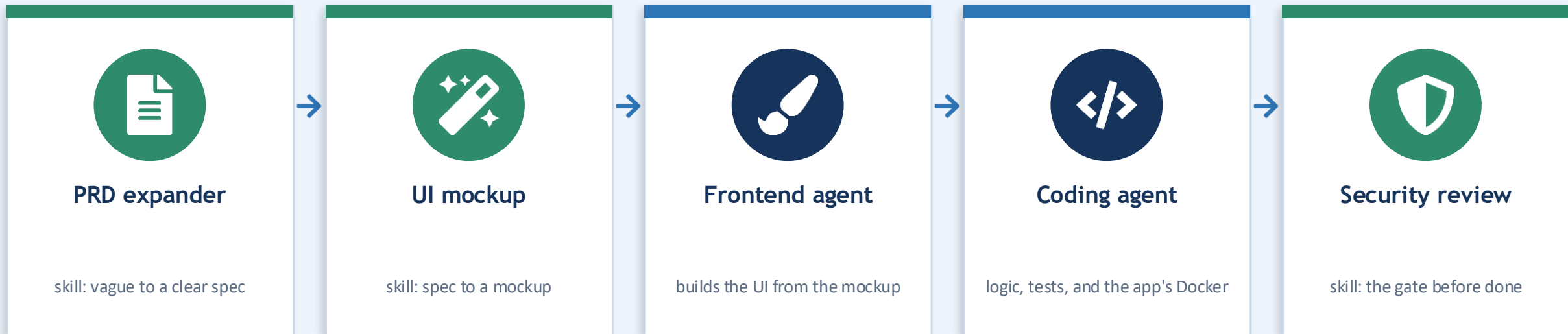
Security review

Finished code to a checklist gate before "done".



From One Line to Shipped Code

One line in, a working app out. Skills (green) and agents (blue) hand off in turn, and you confirm at each step.



Your job changes: you direct and review the team instead of typing every line. The skills enforce quality at each handoff.



THE SKILL THAT EARNS ITS KEEP

Security Review Skill

A checklist the agent runs on its own code before saving it. You don't need these memorised, the skill does the checking. Here's the why; we demo the first two live.



Prompt injection

Text the AI reads (an item note, a vote option) can try to hijack it. Separate data from instructions.



Validate model output

The model returns JSON. Check the schema and ranges before it touches the database.



Input validation

Server-side checks on dates, quantities, everything from the client.



Injection & XSS

Stop user input from running as code or HTML (SQL injection, XSS).



Secrets

No hardcoded keys or tokens. Keys stay server-side.



Error handling

No stack traces or internals leaked to users.

07

SEGMENT

Rapid Build Round — Labs 3 & 4



Same agents-and-skills workspace. Point the same toolkit at two brand-new apps and watch them come together fast.



REUSE IS THE WHOLE POINT

Two More Apps, Far Less Time

The pantry took a full lab. With the same agents and skills, the next apps take a fraction of it, even though they are nothing alike.



Bekantan Rooftop Run

Lab 3 · frontend-led

A one-screen canvas game: a bekantan hopping the rooftops of Kampong Ayer, collecting bananas. The frontend design agent's playground. No backend.

Reuses: UI mockup + design agent + security review



Makan Mana

Lab 4 · security is the star

A group vote: create a poll, share a link, friends vote, results live. The security demo: IDOR on share links, vote tampering, XSS in options.

Reuses: PRD expander + both agents + security review

08

SEGMENT

Side by Side, Takeaways & Q&A



Three apps, one toolkit. What changed, what stuck, and where to go next.



Plain Prompting vs. Agents & Skills

	Plain prompting	Agents & Skills
Workflow	You type every instruction	You direct agents and review
Context	Re-explained each time	Held by the agent's setup
Consistency	Drifts as it grows	Enforced by reusable skills
Security	Easy to forget	A skill checks every time
A second app	Start over from scratch	Reuse the skills, build it fast
Scales to a team	Depends on who is typing	Depends on the playbook



IF YOU REMEMBER FIVE THINGS

Key Takeaways



Agentic coding is a loop

Tools, context, and standing instructions, working until done. Not one prompt and hope.



Agents are specialised teammates

Subagents with a clear scope, a system prompt, and the right tools.



Skills are reusable playbooks

A SKILL.md procedure; now a shared standard across Claude, Codex, and Gemini.



Reuse compounds

Three different apps, one toolkit. The second and third came fast.



Put security in the loop

A review skill catches prompt injection and unvalidated output before they ship.



TAKE THESE HOME

Golden Rules

1

Start vague? Expand it into a PRD first.

2

Agree the UI mockup before wiring any logic.

3

Let the right subagent do the right job, backend vs frontend.

4

Run security-review before you call anything done.

5

Typed roughly the same prompt three times? Promote it to a skill.

selamat ngoding!



Thank you

Questions, comparisons, and war stories welcome.

STAY IN TOUCH



Name: Izdiyar Sulaiman



Email: me@izdiwho.com



Website: izdiwho.com



LinkedIn: linkedin.com/s/izdiyar-s

WORKSHOP RESOURCES



Slides, cheatsheet & starter repo:

github.com/bcsabn/stack-workshop



SEGMENT

Appendix — Files Across Platforms



The same ideas in Claude Code, Codex, and Gemini CLI. Reference, and on your cheatsheet.



The Main Instruction File

Every harness has one always-on file of standing instructions for the main assistant. The name and location differ.

	Claude Code	Codex	Gemini CLI
Main file	CLAUDE.md	AGENTS.md	GEMINI.md
User level	~/.claude/CLAUDE.md	~/.codex/AGENTS.md	~/.gemini/GEMINI.md
Project level	./CLAUDE.md or .claude/	./AGENTS.md (+ nested)	GEMINI.md (+ parents)
Reads AGENTS.md?	No (import or symlink)	Yes, natively	Configurable

AGENTS.md is an open standard (agents.md), now read by Codex and dozens of other tools. Claude Code sticks to CLAUDE.md; Gemini can opt in via a setting.

Sources: code.claude.com/docs · developers.openai.com/codex · geminicli.com/docs · agents.md



Subagent Files Across Platforms

Remember: "agents" in this workshop means subagents. Here's where each harness keeps them.

	Claude Code	Codex	Gemini CLI
Project dir	<code>.claude/agents/</code>	<code>.codex/agents/</code>	<code>.gemini/agents/</code>
User dir	<code>~/.claude/agents/</code>	<code>~/.codex/agents/</code>	<code>~/.gemini/agents/</code>
File format	Markdown + YAML	TOML	Markdown + YAML
Required fields	name, description	name, description, developer_instructions	name, description

Watch out: Claude and Gemini use Markdown with a body that is the system prompt; Codex uses TOML and requires a `developer_instructions` field. Same idea, different syntax.

Sources: code.claude.com/docs/sub-agents · developers.openai.com/codex/subagents · geminikli.com/docs/core/subagents



Skills Across Platforms

Good news: all three now follow the same Agent Skills open standard (agentskills.io), a SKILL.md folder. A rare convergence.

	Claude Code	Codex	Gemini CLI
Project	<code>.claude/skills/<name>/</code>	<code>.agents/skills/</code>	<code>.gemini/skills/</code>
User	<code>~/.claude/skills/</code>	<code>~/.agents/skills/</code>	<code>~/.gemini/skills/</code>
The file	SKILL.md	SKILL.md	SKILL.md
Standard	agentskills.io	agentskills.io	agentskills.io

Notes: Claude also ships skills via plugins and merged its old slash-commands into skills. Codex and Gemini also recognise the shared `.agents/skills/` path. Progressive disclosure everywhere: only the name and description load until a skill is actually used.

Sources: code.claude.com/docs/skills · developers.openai.com/codex/skills · geminicli.com/docs/cli/creating-skills